

Contents

1. Tree Algorithms	2
1.1. What is a Tree?	2
1.2. Subordinates	4
1.3. Tree Matching	6
2. $\text{best}(c1) + \text{best}(c2) + \text{best}(c3)$	7
3. pick best child to match,	7

1. Tree Algorithms

This chapter covers tree algorithm problems from CSES. Trees are connected acyclic graphs with n nodes and $n-1$ edges. Key techniques include: **DFS/BFS traversal**, **Dynamic Programming on Trees**, **Binary Lifting** for ancestor queries, **LCA** (Lowest Common Ancestor), **Euler Tour** for subtree queries, and **Rerooting** technique.

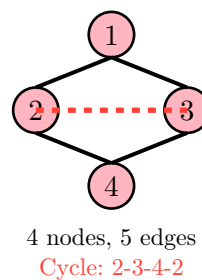
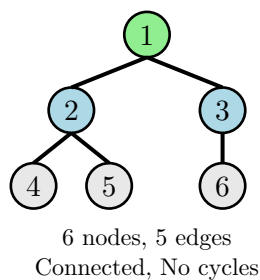
1.1. What is a Tree?

A **tree** is a special type of graph with unique properties that make it fundamental in computer science:

Tree vs Non-Tree Comparison

Valid Tree

Not a Tree (has cycle)

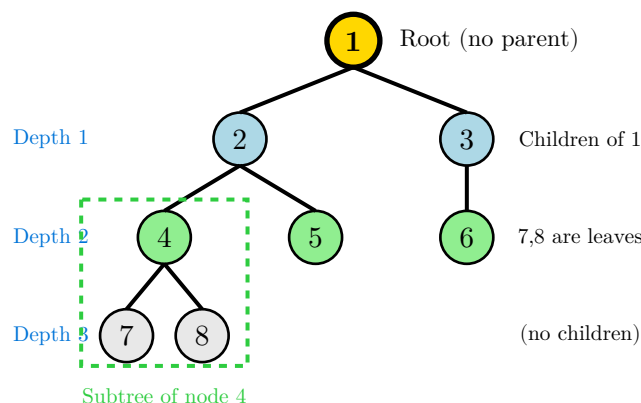


Key Properties of Trees:

- **n nodes, $n-1$ edges:** A tree with n nodes always has exactly $n-1$ edges
- **Connected:** There is a path between every pair of nodes
- **Acyclic:** No cycles exist - exactly one path between any two nodes
- **Rooted vs Unrooted:** We can designate any node as the “root” to give direction

Tree Terminology:

Tree Terminology



- **Root:** The topmost node (node 1 in rooted trees)
- **Parent:** The node directly above (node 2's parent is node 1)
- **Children:** Nodes directly below (node 2's children are 4 and 5)
- **Leaf:** A node with no children (nodes 7, 8, 5, 6)
- **Subtree:** A node and all its descendants (subtree of 4 includes 4, 7, 8)

- **Depth:** Distance from root (root has depth 0)
- **Height:** Maximum depth of any node in the tree

Common Tree Representations in Code:

```
1 // Adjacency List (most common for trees)
2 vector<vector<int>> adj(n + 1); // adj[u] = list of neighbors of u
3
4 // For rooted trees, often store parent separately
5 vector<int> parent(n + 1);
6
7 // Reading a tree with n-1 edges
8 for (int i = 0; i < n - 1; i++) {
9     int a, b;
10    cin >> a >> b;
11    adj[a].push_back(b);
12    adj[b].push_back(a); // undirected
13 }
```

C++

1.2. Subordinates

[Question - Subordinates](#)

[Backup Link](#)

Explanation :

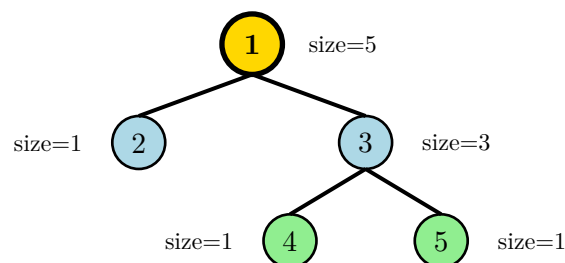
Concepts used: **DFS, Subtree Size Calculation**

Given a company with n employees where each employee (except the boss) has exactly one direct boss, we need to find the number of subordinates for each employee. The boss is employee 1.

This is essentially asking: for each node, how many nodes are in its subtree (excluding itself)?

Key Insight: The number of subordinates of a node = (size of its subtree) - 1. We can compute subtree sizes using a single DFS traversal from the root.

$$\text{Subordinates} = \text{Subtree Size} - 1$$



Subordinates:

Node 1: $5-1 = 4$ subordinates
Node 2: $1-1 = 0$ subordinates
Node 3: $3-1 = 2$ subordinates
Nodes 4,5: $1-1 = 0$ subordinates

Algorithm:

1. Build the tree from parent information
2. Run DFS from root (node 1)
3. For each node, subtree size = 1 + sum of children's subtree sizes
4. Answer for each node = subtree_size - 1

DFS Traversal Order:

```
1 DFS(1):
2   DFS(2): leaf, size[2] = 1
3   DFS(3):
4     DFS(4): leaf, size[4] = 1
5     DFS(5): leaf, size[5] = 1
6     size[3] = 1 + size[4] + size[5] = 3
7   size[1] = 1 + size[2] + size[3] = 5
```

Time Complexity: $O(n)$ - each node visited exactly once

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  vector<vector<int>> children;
6  vector<int> subtree_size;
7
8  int dfs(int u) {
9      subtree_size[u] = 1; // Count self
10     for (int child : children[u]) {
11         subtree_size[u] += dfs(child);
12     }
13     return subtree_size[u];
14 }
15
16 int main() {
17     ios::sync_with_stdio(false);
18     cin.tie(nullptr);
19
20     cin >> n;
21     children.resize(n + 1);
22     subtree_size.resize(n + 1);
23
24     // Read parent of each employee (except boss)
25     for (int i = 2; i <= n; i++) {
26         int parent;
27         cin >> parent;
28         children[parent].push_back(i);
29     }
30
31     // DFS from root to compute subtree sizes
32     dfs(1);
33
34     // Output subordinates (subtree_size - 1)
35     for (int i = 1; i <= n; i++) {
36         cout << subtree_size[i] - 1;
37         if (i < n) cout << " ";
38     }
39     cout << "\n";
40
41     return 0;
42 }
```

1.3. Tree Matching

[Question - Tree Matching](#) [Backup Link](#)

Explanation :

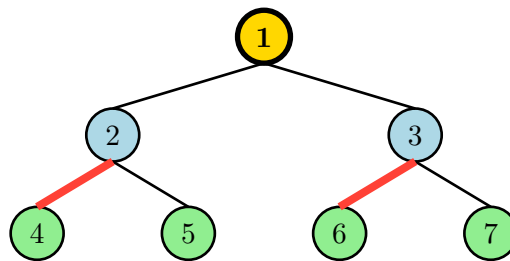
Concepts used: **Tree DP** (Dynamic Programming on Trees)

Given a tree with n nodes, find the maximum matching - the largest set of edges where no two edges share a common node. Each node can be part of at most one matched edge.

Key Insight - Tree DP: For each node, we track two states:

- $dp[u][0]$ = max matching in subtree of u , where u is **not** matched with any child
- $dp[u][1]$ = max matching in subtree of u , where u **is** matched with one of its children

Tree Matching Example



Maximum Matching = 2 edges

Matched: (2,4) and (3,6)

Could also be (2,5) and (3,7)

DP Transitions:

Case 1: Node u is NOT matched ($dp[u][0]$)

- All children can be in either state (matched or unmatched)
- Take the best option for each child

```
1 dp[u][0] = sum over children c of max(dp[c][0], dp[c][1])
```

Case 2: Node u IS matched with one child ($dp[u][1]$)

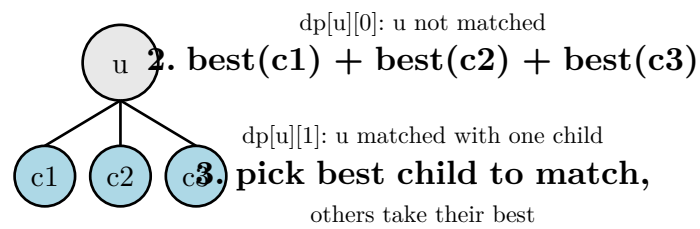
- Pick one child c to match with u (adds 1 edge)
- That child must be unmatched ($dp[c][0]$) since we're using it
- Other children take their best state

```
1 dp[u][1] = max over children c of:
2     (dp[c][0] + 1) + sum over other children c' of max(dp[c'][0], dp[c'][1])
```

This simplifies to:

```
1 dp[u][1] = dp[u][0] + max over children c of (dp[c][0] + 1 - max(dp[c][0], dp[c][1]))
```

DP State Visualization



Time Complexity: $O(n)$ - each node processed once

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  vector<vector<int>> adj;
6  vector<array<int, 2>> dp; // dp[u][0], dp[u][1]
7
8  void dfs(int u, int parent) {
9      dp[u][0] = 0; // u not matched
10     dp[u][1] = 0; // u matched with a child
11
12     for (int v : adj[u]) {
13         if (v == parent) continue;
14         dfs(v, u);
15
16         // For dp[u][0]: take best of each child
17         dp[u][0] += max(dp[v][0], dp[v][1]);
18     }
19
20     // For dp[u][1]: try matching u with each child
21     for (int v : adj[u]) {
22         if (v == parent) continue;
23
24         // Match u with v: gain 1 edge, v must be unmatched
25         // Other children already counted in dp[u][0]
26         int gain = dp[v][0] + 1 - max(dp[v][0], dp[v][1]);
27         dp[u][1] = max(dp[u][1], dp[u][0] + gain);
28     }
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
```

C++

```

34
35     cin >> n;
36     adj.resize(n + 1);
37     dp.resize(n + 1);
38
39     for (int i = 0; i < n - 1; i++) {
40         int a, b;
41         cin >> a >> b;
42         adj[a].push_back(b);
43         adj[b].push_back(a);
44     }
45
46     dfs(1, 0);
47
48     cout << max(dp[1][0], dp[1][1]) << "\n";
49
50     return 0;
51 }

```